

SketchLang

A Mini Domain-Specific Language for Vector Graphics Generation

Compiler Design Laboratory Project — Phase 2: Core Implementation & Progress Report

Student Name	Nandana Krishnan
Register No.	24BCE2081
Course	Compiler Design Lab
Project Type	Individual
Phase	Phase 2 of 3 — Core Implementation
Test Status	59 / 59 automated unit + integration tests passing (96% statement coverage)

1. Phase 2 Objective

Phase 1 delivered a single-file prototype covering the full compiler pipeline for a fixed-literal version of SketchLang. The objective of Phase 2 is to turn that prototype into a properly modularized implementation, add a genuine language feature that requires real compiler-theory work — variables backed by a symbol table, with arithmetic expressions — add a second, independent optimization, and back the whole system with an automated test suite.

2. What Changed Since Phase 1

- The single `sketchlang.py` file was split into eight focused modules (Section 3), each independently testable.
- Added variables: “let name = expr”, with a real symbol table and undefined-variable checking.
- Added a small expression grammar: +, -, *, /, unary minus, and parentheses, with correct precedence.
- Fixed a real grammar ambiguity: “move x y” could not tell two negative numbers from a subtraction; “move” now requires a comma between its two coordinates (“move x, y”).
- Added a second optimization pass, dead-move elimination, independent of the Phase 1 collinear-line merge.
- Replaced the fixed 400×400 canvas with an auto-fit SVG `viewBox` computed from the drawing's bounding box.
- Added 59 automated unit and integration tests (unittest) covering every module, reaching 96% statement coverage.

3. Core Implementation Modules

The Phase 1 prototype is now split as follows. Each module has a single, clearly-named responsibility:

Module	Responsibility
<code>lexer.py</code>	Regex-based tokenizer: keywords, identifiers, numbers, operators, punctuation, comments; raises <code>LexError</code> with a line number on an invalid character.
<code>parser.py</code>	Recursive-descent parser implementing the full grammar (statements + expressions); builds the AST; raises <code>ParseError</code> on malformed input.
<code>symboltable.py</code>	<code>SymbolTable</code> class: name → (value, declaration line); supports <code>declare/lookup/is_declared</code> .
<code>semantics.py</code>	Static, single-pass semantic checker (colors, undefined variables, repeat counts, division by zero) and the shared <code>evaluate_expr()</code> used by both semantics and IR generation.
<code>ir.py</code>	Iteration-aware AST executor: turns the AST into a flat instruction list, correctly re-evaluating variables on every loop pass.
<code>optimizer.py</code>	Two independent peephole passes: <code>eliminate_dead_moves</code> and <code>merge_collinear_lines</code> .
<code>codegen.py</code>	Emits SVG from the optimized IR, with an automatically computed bounding-box <code>viewBox</code> .

compiler.py	Driver: wires the six stages above together behind one compile_source() function.
-------------	---

main.py provides the command-line interface (compile a file, optionally print the IR before/after optimization and the final symbol table).

4. Module-wise Explanation of Key Design Decisions

4.1 The Expression Grammar and a Real Ambiguity

Supporting variables meaningfully required a proper expression grammar, not just single numeric literals. The grammar ($\text{expr} \rightarrow \text{term} (('+' | '-') \text{term})^*$, $\text{term} \rightarrow \text{factor} (('*' | '/') \text{factor})^*$, $\text{factor} \rightarrow \text{NUMBER} \mid \text{IDENT} \mid '(' \text{expr} ')' \mid '-' \text{factor}$) is a standard precedence-climbing design that correctly parses “2 + 3 * 4” as 2 + (3 * 4). Implementing this exposed a genuine ambiguity: “move -5 -5” could be read either as two negative numbers or as one subtraction expression ((-5) - 5). Rather than special-casing it, “move” now requires a comma between its two arguments — the same fix real languages use for this exact class of ambiguity in argument lists.

4.2 Two Passes Over the Same Program, On Purpose

Semantic checking and IR generation both walk the AST, but for different reasons, and this distinction is intentional. check_semantics() makes a single static pass, in textual order, to catch mistakes (bad colors, undefined variables, invalid repeat counts) before any output is generated — a repeat body is visited once, not once per iteration, exactly as a real compiler's semantic-analysis phase would. generate_ir() then walks the AST again, this time as a full execution: its symbol table is live and is updated on every pass through a repeat loop. Both passes share one evaluate_expr() function (in semantics.py) so that “what an expression means” is defined in exactly one place.

4.3 Why That Distinction Is Visible in the Spiral Example

examples/spiral.sketch reassigns step with “let step = step + 2” inside a 40-times repeat. The static semantic checker's symbol table only reflects one pass through the loop body, so after check_semantics() runs, step is 6, not its final value. The IR generator's own symbol table, however, is updated all 40 times, which is why the compiled picture is an actual spiral of 40 different segment lengths rather than 40 identical ones. This is demonstrated in Section 6.2 and is a direct, demonstrable consequence of the two-pass design above.

4.4 Two Independent Optimizations

eliminate_dead_moves() removes a MOVE instruction that is immediately followed by another MOVE, since only the pen's final position before the next drawn line matters. merge_collinear_lines() (carried over from Phase 1) merges two consecutive same-color LINE instructions when the first segment's endpoint is the second segment's start point and all three points are collinear. optimize_ir() runs them in sequence; both are proven, in test_optimizer.py, to never increase the instruction count and to never change the picture the program draws.

4.5 Auto-fit Output Canvas

The Phase 1 prototype used a fixed 400×400 canvas with a constant margin, which does not scale to larger drawings such as the spiral. codegen.py now computes the bounding box of every point the optimized IR actually visits and sizes the SVG viewBox to fit it with a small fixed margin, so any program — tiny or large — is framed correctly without the user specifying a canvas size.

5. Test Cases

59 automated tests were written with Python's unittest, organized one file per module plus an integration suite that runs the full compiler pipeline end-to-end on the example programs (including the three error-case programs). Statement coverage across the sketchlang package is 96% (measured with coverage.py).

Test file	What it covers
test_lexer.py	Token kinds for every statement type, decimals, keywords, comments/whitespace skipping, line-number tracking, comma/parentheses, and the LexError path for an invalid character.
test_parser.py	Operator precedence ($2+3*4$ vs $(2+3)*4$), unary minus, let statements, the required move comma (and that omitting it is a ParseError), repeat blocks, and unterminated/garbled input.
test_semantics.py	A valid program; unknown colors; undefined variables; non-positive and non-integer repeat counts; division by zero; correct expression evaluation; variable reassignment.
test_ir.py	Straight-line and turned movement geometry, MOVE vs LINE emission, repeat expansion, variable-driven distances, and — the key Phase-2 test — a variable that grows by 1 on every one of 3 loop iterations.
test_optimizer.py	Dead-move collapsing, collinear-line merging (and correctly NOT merging on color/direction/gap mismatches), and that optimization never increases instruction count.
test_codegen.py	One <line> element per LINE instruction, MOVE producing no visible element, well-formed SVG, and the viewBox growing for larger drawings.
test_integration.py	compile_source() end-to-end on all seven example programs, including the three deliberately invalid ones, and that each error stage (lexical/syntax/semantic) actually stops the pipeline.

5.1 Full Test Run

```
$ python3 -m unittest discover -s tests -v
...
-----
Ran 59 tests in 0.003s

OK
```

5.2 Coverage Report

Name	Stmts	Miss	Cover	Missing

sketchlang/__init__.py	5	0	100%	
sketchlang/codegen.py	26	0	100%	
sketchlang/compiler.py	26	0	100%	
sketchlang/ir.py	42	0	100%	
sketchlang/lexer.py	33	2	94%	55, 58
sketchlang/optimizer.py	24	1	96%	45
sketchlang/parser.py	100	2	98%	38, 159
sketchlang/semantics.py	58	5	91%	58, 64-67, 111
sketchlang/symboltable.py	21	5	76%	34, 42, 46, 49, 52

TOTAL	335	15	96%	

The uncovered lines are almost entirely defensive branches (e.g. an unreachable 'unknown operator' fallback) rather than untested feature logic.

6. Functional Demonstration & Intermediate Results

6.1 Variables and Arithmetic Expressions

examples/variables.sketch declares a side length, draws a square with it, then derives a second square's side as an expression (side * 2):

```
# Phase 2: squares sized using variables and arithmetic expressions.
let side = 80
pen blue
repeat 4 {
  forward side
  turn 90
}

let bigSide = side * 2
move (side * 2), 0
pen orange
repeat 4 {
  forward bigSide
  turn 90
}
```

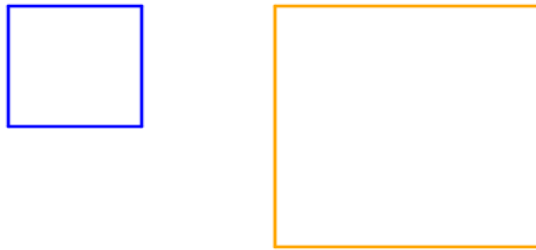


Figure 1: two squares, the second sized entirely from an arithmetic expression over the first's variable. The symbol table after compilation reports side = 80 and bigSide = 160, confirming the expression was evaluated correctly.

6.2 A Spiral: Variables That Change Every Loop Iteration

examples/spiral.sketch is the clearest evidence that the symbol table is live during code generation, not just checked once:

```
# Phase 2: a growing spiral -- "step" is reassigned every loop iteration,
# which only works because the symbol table is updated live during IR
# generation, not just checked once statically.
let step = 4
pen purple
repeat 40 {
  forward step
  turn 20
  let step = step + 2
}
```

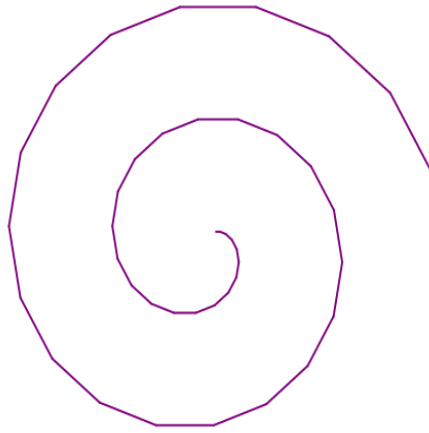


Figure 2: the compiled spiral. step starts at 4 and grows by 2 on every one of the 40 repeat iterations, so each of the 40 LINE instructions in the IR has a different length — something that is only possible because ir.py's SymbolTable is updated on every pass through the loop body, not evaluated once statically.

6.3 Dead-Move Elimination (New in Phase 2)

Three consecutive “move” calls with no drawing in between are dead code — only the last one before the next line matters:

```
move 10, 10
move 50, 50
move 100, 100
pen black
forward 40

> Unoptimized IR: 4 instruction(s)
  ('MOVE', 10.0, 10.0)
  ('MOVE', 50.0, 50.0)
  ('MOVE', 100.0, 100.0)
  ('LINE', 100.0, 100.0, 140.0, 100.0, 'black')
> Optimized IR: 2 instruction(s)
  ('MOVE', 100.0, 100.0)
  ('LINE', 100.0, 100.0, 140.0, 100.0, 'black')
```

6.4 New Semantic Error Cases

Two new categories of semantic error, both caught before any IR or SVG is generated:

```
pen red
forward radius      # 'radius' was never declared with 'let'

> Compilation failed: Semantic error (line 2): variable 'radius' used before it is declared

let a = 10
let b = 0
forward a / b      # division by zero inside an expression

> Compilation failed: Semantic error (line 3): division by zero
```

7. Implementation Progress Report (Review 2 Self-Assessment)

Review 2 Criterion	Status this phase
Implementation Progress	All Phase 2 planned work is complete: modularization, variables/symbol table, expression grammar, a second optimization pass, and an auto-fit code generator.
Functional Correctness	Verified by 59 passing automated tests plus manual inspection of rendered SVG output for every example program.
Compiler Concept Application	Symbol table management and a real expression grammar were added on top of the Phase 1 lexer/parser/semantics/IR/optimizer/codegen pipeline.
Code Quality	Single-responsibility modules, docstrings explaining the purpose and design choice of each stage, and no remaining single-file prototype code.
Testing	unittest-based suite, one file per module plus an integration suite; 96% statement coverage measured with coverage.py.
Problem Solving	Identified and resolved a genuine grammar ambiguity in the 'move' statement (adjacent unary-minus vs. subtraction) via a small, well-justified grammar change.
Individual Understanding	Every design decision above (two-pass semantics vs. IR generation, the two optimizations, the comma fix) is documented with the reasoning behind it for the viva.

8. Remaining Work for Phase 3

- Add a second target backend (e.g. a simple bytecode + interpreter, or PNG raster export) to demonstrate retargetability.
- Add boundary and stress test cases: deeply nested repeats, very large coordinates, malformed/unbalanced braces.
- Measure and report compilation time and IR-size reduction from optimization across a larger set of sample programs.
- Consider small originality additions: named re-usable procedures, or a second shape primitive (e.g. filled polygons).
- Finalize the complete project report, user instructions, and prepare the live Review 3 demonstration.

REPO LINK: <https://github.com/glitched-nk/sketchLang>